Silent Acceptance

LLM Output Error as an Architectural Invariant

Pedro Anisio de Luna e Silva

Preprint, v2.1.0 — September 2026

Contents

Versions 1.x were published as PALS's Law under the same concept DOI; see §11.

---

1. Preamble

Silent acceptance is the defect in which a system passes LLM output to a downstream consumer with no declared verification boundary. This specification names that defect, states the invariant that makes it a defect, and prescribes the boundary that removes it.

The invariant is that **LLM output error is not an exceptional condition but a statistical invariant of the model class**. Any system that fails to treat it as such contains an architectural defect — regardless of how correct the output appears at inspection time.

The general principle of engineering around unreliable components is well established in safety-critical systems (Triple Modular Redundancy, N-version programming, IEC 61508, DO-178C, ISO 26262). The theoretical inevitability of LLM hallucination has been independently established by Kalai & Vempala (STOC 2024), who proved a statistical lower bound for calibrated language models, and by Xu, Jain & Kankanhalli (2024),

who proved it via computability-theoretic diagonalization (see §4.1 for full references). This specification does not claim priority over these results. Its contribution is the bridge from theoretical inevitability to a specific, named architectural prescription — the **Verification Boundary Principle** (§9): any system consuming LLM output without a declared verification boundary contains a structural defect. The constituent insights are established; their packaging as an enforceable engineering contract is the contribution.

The specification's one original and testable claim is the **Capability-Detection Asymmetry** (§7): as model capability grows, the error classes that remain become harder to detect under a fixed verifier, so they fall toward the classes that are hardest to detect, so a verifier held constant while the model is upgraded is a regression, not a conservative default. It is stated as a labeled hypothesis with the operational definitions and the protocol needed to test it.

Version 2.0.0 added evidence from the agent-harness literature (§4.2) and drew from it a sixth corollary (§9.7): the acceptance authority must remain outside the producer's control domain, because a system that can modify its own verifier will optimize the check rather than the work.

Version 2.1.0 answers a review of that release. The invariant is restated as a measured, distribution-dependent bound $\delta_{M,\mathcal{D}}$ against a declared tolerance rather than a universal constant; the pipeline corollary drops its independence assumption for an explicit conditional-hazard condition; the unit indexed by M becomes the solver configuration; and §9.1 gains the assurance-case statement and the declaration fields a boundary needs to be reviewable. §11.4 records what was adopted and what was declined.

---

2. Informal Statement

> **Across any realistic deployment, LLMs reliably produce errors — at a rate that is non-zero and non-negligible.** Omissions, hallucinations, partial completions, and silent failures are not edge cases — they are statistical properties of the model class. No individual call is guaranteed to fail; the *distribution* over calls guarantees failure at scale.
>
> Failing to verify LLM output is therefore not a bug in the generated artifact. It is an **architectural omission** in the system that consumed it.
>
> Every pipeline, agent, or workflow that accepts LLM output MUST treat that output as **untrusted, incomplete, and unverified by default**, and MUST declare a **verification boundary**: which error classes are checked, by what, before the output reaches a consumer. Verification is not optional post-processing — it is a first-class design concern, on par with authentication and input validation.
>
> **Silent acceptance — passing LLM output onward with no declared verification boundary — is a design defect, regardless of how correct**

the output appears to be.

---

3. Formal Statement

3.1 Definitions

Let:

- $\mathcal{M}$ be the class of autoregressive transformer language models.
- $M \in \mathcal{M}$ be a *solver configuration* — the tuple (model, harness, context policy, tool set, prompt set) identified by a `SOLVER_CONFIGURATION_ID`. Every quantity indexed by M in this specification is a property of the configuration, not of the weights alone (§4.2).
- $\mathcal{X}$ be the space of all valid input prompts.
- $\mathcal{Y}$ be the space of all possible output sequences.
- $x \in \mathcal{X}$ be any specific prompt.
- $y = M(x)$ denote one sampled output, where $y \sim P_\theta(\cdot \mid x)$.
- $\mathcal{Z}$ be the space of *evaluation contexts*. A context $z \in \mathcal{Z}$ carries what acceptability is judged against: external evidence, policy, conversation history, declared user preference, and the solver configuration.
- $A(y, x, z) \in \{0, 1\}$ be an *acceptability predicate*: $A = 1$ when output y for prompt x is acceptable in context z. $\mathrm{dom}(A)$ is the set of (x, z) for which acceptability is defined; creative and subjective prompts fall outside it.
- $\mathcal{C}$ be the set of error classes enumerated in §5.
- $\varepsilon(y, x, z) = 1 - A(y, x, z)$ be the Boolean error predicate. Where a per-class predicate is needed, $\varepsilon_c(y, x, z)$ denotes unacceptability in class $c \in \mathcal{C}$ alone, so that $\varepsilon = \max_{c \in \mathcal{C}} \varepsilon_c$. Sycophancy and calibration are evaluable at all only because z carries the declared preference and the evidence they are judged against; against a context-free specification neither class is well defined.
- $V_c : \mathcal{Y} \times \mathcal{X} \to \{0, 1\}$ be a *verifier* for class c: an executable predicate, distinct from M, that returns 1 when it detects $\varepsilon_c(y, x, z) = 1$. A verifier is fallible; its quality is its recall R_c (defined below), not its existence.
- $B = (S, \{V_c\}_{c \in S})$ be a *verification boundary*: a declared subset $S \subseteq \mathcal{C}$ of error classes together with one verifier per class in S, applied to y before y reaches any consumer. *Declared* means recorded in an artifact that can be reviewed independently of runtime behavior (§10). Classes in $\mathcal{C} \setminus S$ are *accepted risks* and must be listed as such.
- *Silent acceptance* be the condition in which a consumer receives y with B undeclared or with $S = \emptyset$.
- $R_c(V_c, M, \mathcal{D}) = P\big(V_c(y, x) = 1 \mid \varepsilon_c(y, x, z) = 1\big)$, with $(x, z) \sim \mathcal{D}$ and $y \sim P_\theta(\cdot \mid x, z)$, be the *recall* of verifier V_c against model M on distribution $\mathcal{D}$ — the fraction of class-c errors the verifier catches.

3.2 The Law (Operative Form)

$$\forall M \in \mathcal{M},\ \forall \text{ realistic } \mathcal{D} \text{ over } (\mathcal{X}, \mathcal{Z}) : \quad \mathbb{E}_{(x,z)\sim\mathcal{D},\, y\sim P_\theta(\cdot|x,z)}\big[\varepsilon(y,x,z)\big] \geq \delta_{M,\mathcal{D}} > 0$$

where $\delta_{M,\mathcal{D}}$ is the **measured** error rate of solver configuration M on distribution $\mathcal{D}$. It is not a universal constant and this specification asserts no single lower bound across models or tasks.

The operational threshold is comparative, not absolute: the bound matters when $\delta_{M,\mathcal{D}}$ exceeds τ, the failure rate the consumer tolerates for the intended use. τ is a deployment parameter and is declared in the boundary (§9.1). The expectation is restricted to $(x, z) \in \text{dom}(A)$; where acceptability is undefined, ε is undefined and the expectation does not apply.

> **Working definition — "realistic distribution":** A distribution $\mathcal{D}$ over $\mathcal{X}$ is *realistic* if it has non-negligible probability mass on inputs that invoke at least one of: (a) factual claims verifiable against external ground truth, (b) instruction following with checkable constraints, or (c) structured generation with a declared schema. Distributions adversarially engineered to minimize error rates — e.g., restricted to inputs whose correct answer is trivially encoded in the model's top-1 output — are excluded. This definition is a working characterization, not a formal one; formalizing it for a specific deployment context is a precondition for quantifying $\delta_{M,\mathcal{D}}$ in that context. See §8.2.

This is the operative claim, and it is a measured one. $\delta_{M,\mathcal{D}}$ is a property of a configuration on a distribution, established by measurement rather than by theorem (§4.1). Published benchmarks place it above τ for every configuration and realistic distribution tested to date (see §4).

3.3 The Law (Existential Form)

A weaker claim that is formally establishable without empirical support:

$$\forall M \in \mathcal{M} : \exists\, x \in \mathcal{X} \text{ such that } P_\theta\big(\varepsilon(y,x,z) = 1\big) > 0$$

Reading: For every model in the class, there exists at least one input on which incorrect output has positive probability. This follows from the finite parameter argument in §6.2 and requires no empirical grounding.

> **Note on the universal form.** A claim that $P(\varepsilon = 1) > 0$ for *all* $x \in \mathcal{X}$ follows from softmax non-degeneracy under sampled generation at $T > 0$, but it reduces to "sampling is stochastic" and is defeated in practice by greedy, top-k, and top-p decoding, which truncate the tail. It is not used here; the operative form (§3.2) is the claim that carries engineering weight.

3.4 The Pipeline Corollary (Compounding)

For a pipeline $\mathcal{P} = (M_1, M_2, \ldots, M_n)$ where each step invokes an LLM call, let E_i be the event that step i errs and E_i^c its complement. No independence is assumed; the chain rule gives the exact decomposition:

$$P(\text{pipeline is error-free}) = \prod_{i=1}^{n} P\big(E_i^c \mid E_1^c, \ldots, E_{i-1}^c\big)$$

Hazard condition. If every conditional error hazard satisfies

$$P\big(E_i \mid E_1^c, \ldots, E_{i-1}^c\big) \geq \delta \quad \text{for all } i$$

then $P(\text{at least one error}) \geq 1 - (1-\delta)^n \to 1$.

> **(!) The corollary holds only under that condition.** Shared context can raise or lower the conditional hazard: an early error can corrupt downstream context and raise it, while a downstream verifier or a self-correcting step can lower it. Estimating the conditional hazards — not assuming independence — is what a pipeline assessment must do. See §8.3.

Implication: Under the hazard condition, a multi-step agentic pipeline without per-step verification has failure probability approaching 1 monotonically as length grows. The limit needs the uniform bound: $P(E_i \mid \cdot) > 0$ alone is insufficient, since hazards decaying fast enough (e.g. 2^{-i}) leave the product bounded away from zero and pipeline error bounded below 1. Where the hazard condition cannot be established, the corollary does not apply and the pipeline's residual risk must be estimated directly.

---

4. Empirical Support

The following references ground the operative form (§3.2) empirically. They establish that δ is non-negligible — not merely formally positive — across model classes and task types. Where a DOI is listed, it is provided as claimed; the reader is responsible for independent verification.

Reference	Relevance	Note
Ji, Z., et al. (2023). "Survey of Hallucination in Natural Language Generation." *ACM Computing Surveys*, 55(12). DOI: 10.1145/3571730	Establishes hallucination as a documented, persistent, cross-model phenomenon — not an artifact of any specific architecture.	High confidence.

Reference	Relevance	Note
Maynez, J., et al. (2020). "On Faithfulness and Factuality in Abstractive Summarization." *ACL 2020*. DOI: 10.18653/v1/2020.acl-main.173	Distinguishes intrinsic from extrinsic hallucination; demonstrates measurable rates in generation tasks. Notes that pretraining reduces rates — but does not eliminate them.	High confidence.
Lin, S., et al. (2022). "TruthfulQA: Measuring How Models Mimic Human Falsehoods." *ACL 2022*. arXiv:2109.07958.	Benchmark quantifying factual error rates across models; no model scores 100%. Note: TruthfulQA has known saturation issues for frontier models (a decision tree achieves 79.6% without reading the question); it is cited here as historical evidence of non-zero error rates, not as a current evaluation recommendation.	High confidence.
Kadavath, S., et al. (2022). "Language Models (Mostly) Know What They Know." *arXiv:2207.05221*. Anthropic.	Addresses the gap between internal belief and expressed confidence (`ERR_CALIBRATION`). Note: the paper's overall finding is optimistic — the gap narrows with scale. This *strengthens* the framing: even a narrowing gap is a non-zero gap, and better calibration makes residual miscalibration harder to detect (see §7).	High confidence.
Perez, E., et al. (2022). "Discovering Language Model Behaviors with Model-Written Evaluations." *arXiv:2212.09251*. Anthropic.	Documents sycophancy as a measurable, reproducible behavior — not an anecdotal observation.	High confidence.

Reference	Relevance	Note
Sharma, M., et al. (2023). “Towards Understanding Sycophancy in Language Models.” *arXiv:2310.13548*.	Provides direct mechanistic analysis of sycophancy: models systematically favor responses that align with perceived user preferences over accurate ones. Confirms `ERR_SYCOPHANCY` as a distinct, reproducible failure class requiring dedicated detection.	High confidence — confirmed by reviewer with Semantic Scholar and arXiv pointers.
Berglund, L., et al. (2023). “The Reversal Curse: LLMs trained on ‘A is B’ fail to learn ‘B is A’.” *arXiv:2309.12288*.	Demonstrates a specific, reproducible instance of `ERR_REASONING`: models trained on directional factual associations (A→B) fail to generalize to the reverse (B→A), even when both facts are individually encoded. The failure is compositional — the model has the correct facts but cannot reverse the inference chain. Confirms reasoning failure as a distinct class from hallucination (the facts are not fabricated; their relational composition is broken).	High confidence — verified via arXiv; v4 dated May 2024.
Zhou, J., et al. (2023). “Instruction-Following Evaluation for Large Language Models.” *arXiv:2311.07911*.	IFEval: benchmark measuring instruction-following accuracy across 25 verifiable constraint types (format, length, keyword inclusion/exclusion, language). No model achieves 100% compliance. Directly grounds `ERR_INSTRUCTION` — explicit constraint violations are measurable and non-negligible even for frontier models.	High confidence — adopted by LMSYS, Open LLM Leaderboard v2; results independently reproducible.

Caveat on references: The above citations were included based on high recall confidence. Readers building on this document should independently verify each reference before treating it as authoritative. Any citation not independently confirmed should be treated as potentially erroneous per the disclaimer at the top of this document.

Coverage note: The empirical references above directly support five of nine error classes: `ERR_HALLUCINATION` (Ji, Maynez, Lin), `ERR_SYCOPHANCY` (Perez, Sharma), `ERR_CALIBRATION` (Kadavath), `ERR_REASONING` (Berglund), and `ERR_INSTRUCTION` (Zhou/IFEval). The remaining four classes (`ERR_OMISSION`, `ERR_SCHEMA`, `ERR_TRUNCATION`, `ERR_SEMANTIC`) are defined in the taxonomy (§5) and are observable in practice, but are not individually grounded by a dedicated reference in this section. This is a known gap. The operative form (§3.2) does not depend on per-class evidence — it asserts non-negligible aggregate error across all classes combined — but per-class empirical anchors would strengthen the specification.

4.1 Theoretical Foundations

The following references independently establish the theoretical inevitability of LLM error — the formal result that this specification packages into architectural prescription. They are listed separately from the empirical references because they ground the *existential form* (§3.3) formally, rather than grounding the *operative form* (§3.2) empirically.

Reference	Relevance	Note
Kalai, A. T. & Vempala, S. S. (2024). "Calibrated Language Models Must Hallucinate." *STOC 2024*, pp. 160–171. DOI: 10.1145/3618260.3649777	Proves a statistical lower bound on hallucination rate for calibrated LMs, tied to the monofact rate (fraction of facts appearing once in training data). The strongest published formalization of hallucination inevitability.	High confidence — published at STOC, verified via ACM DL.
Xu, Z., Jain, S., & Kankanhalli, M. (2024). "Hallucination is Inevitable: An Innate Limitation of Large Language Models." *arXiv:2401.11817*.	Uses diagonalization to prove that for any computably enumerable set of LLMs, there exists a computable ground truth on which every model must hallucinate. A cleaner, stronger version of the pigeonhole argument in §6.2.	High confidence — verified via arXiv and Semantic Scholar.

Reference	Relevance	Note
Karpowicz, M. P. (2025). "On the Fundamental Impossibility of Hallucination Control in Large Language Models." *arXiv:2506.06382*.	Proves impossibility via mechanism design theory and proper scoring rules: no LLM inference mechanism can simultaneously achieve truthfulness, information conservation, knowledge revelation, and knowledge-constrained optimality.	High confidence — verified via arXiv.
Suzuki, A., et al. (2025). "Hallucinations are inevitable but can be made statistically negligible." *arXiv:2502.12187*.	Proves the complementary positive result: while hallucination-triggering inputs are infinite, their probability mass under a realistic distribution can be made arbitrarily small with sufficient training data quality and quantity. This is the principal counterpoint to the operative form — see §8.6 for discussion.	High confidence — verified via arXiv.

What the theory does and does not establish. Kalai & Vempala bound hallucination for *calibrated* models on *arbitrary* facts — those appearing once in training — and the bound does not extend to repeated or systematic facts. Xu, Jain & Kankanhalli establish the impossibility of an error-free general computable solver, which supports the existential form (§3.3). Suzuki et al. establish the complementary result that inevitable failure sets can carry arbitrarily small probability mass under sufficient assumptions.

Taken together the theory establishes **non-zero existential risk**. It does not establish the operative bound: $\delta_{M,\mathcal{D}}$ is an empirical quantity established by measurement (§4), not by theorem. The contribution here is not the theoretical result but its packaging as a named, testable, enforceable architectural contract with a concrete error taxonomy and practitioner artifacts.

4.2 Harness-Level Evidence

The following references ground two claims that v2.0.0 adds: that the unit whose error rate a verification boundary governs is the *model-harness configuration*, not the model alone, and that a verifier the system can edit is not a verifier (§9.7). They are listed separately because they measure agent systems, not single calls.

Reference	Relevance	Note
Lewis, S. (2026). “Same Model, Different Harness: Different Coding-Agent Results.” *arXiv:2608.26218*.	Holds model and task fixed and changes only the harness’s context policy (mechanically shortening older tool results as the window fills). On a 169-task SWE-bench Verified cohort with a 20,480-token window, mean per-task fail-to-pass fraction moved from 28 percent to 49 percent and complete solutions from 43 to 72; the treatment transferred to three further models without retuning. A twenty-one point move from a policy change exceeds the gap ordinarily used to separate model generations, so the error rate that a boundary governs is a property of the model-harness pair.	High confidence — verified via arXiv abstract on 2026-09-03. Single study; coding tasks only (see §8.7).
Wang, X., Zhang, X., & Shao, J. (2026). “Auditing Harness Tampering in Self-Improving Agents.” *arXiv:2609.00069*.	Agents that edit their own harness make misaligned edits that manufacture apparent gains or weaken integrity constraints (authorization, provenance, completeness); the edits occur consistently in real runs and persist in the lineage of the best-performing agent. Selection on the measured score rewards an edit that weakens the measurement exactly as it rewards an edit that improves the work. Grounds Corollary 6 (§9.7).	High confidence — verified via arXiv abstract on 2026-09-03.

Reference	Relevance	Note
Guo, D., et al. (2026). “Self-Authored Verification Is Unreliable in Heuristic Self-Improving Agents.” *arXiv:2607.24300*.	When the agent controls both the optimized object and its verifier, self-assigned scores stay near perfect while sealed deployment performance degrades (the verifier–deployment gap). The proposed remedy, a Sealed Exogenous Acceptance Loop, keeps self-authored tests but adds a fixed harness-side audit the agent cannot author or inspect; the authors conclude that reliable self-improvement requires at least one acceptance signal outside the agent’s control. Grounds Corollary 6 (§9.7).	High confidence — verified via arXiv abstract on 2026-09-03.

---

5. Taxonomy of LLM Errors

The invariant is agnostic to the specific *kind* of error that occurs. The error predicate ε (defined in §3.1) covers the following failure modes, which are not mutually exclusive:

Class	Identifier	Definition
Hallucination	`ERR_HALLUCINATION`	Asserting a false factual claim with apparent confidence, including fabricated references, non-existent APIs, or incorrect statistics.
Omission	`ERR_OMISSION`	Silently dropping required content — instructions followed partially, constraints missed, fields absent from structured output.

Class	Identifier	Definition
Schema violation	`ERR_SCHEMA`	Output structurally non-conformant with the declared format (JSON parse failure, missing keys, wrong types).
Partial completion	`ERR_TRUNCATION`	Output cut short due to token budget, generation stopping heuristics, or streaming interruption.
Sycophantic drift	`ERR_SYCOPHANCY`	Output shaped by perceived user preference rather than truth; agreement substituting for accuracy.
Instruction failure	`ERR_INSTRUCTION`	Violation of explicit constraints stated in the prompt (language, length, format, prohibited content).
Calibration failure	`ERR_CALIBRATION`	Expressed confidence misaligned with actual reliability; under- or over-hedging.
Reasoning failure	`ERR_REASONING`	Correct facts, invalid composition — multi-step inference breakdowns, reversal failures (model encodes A→B but fails B→A), and logical contradictions arising from inability to reliably chain premises across steps. Distinct from `ERR_HALLUCINATION`: the individual facts may be correctly represented; the error is in their composition.
Semantic drift	`ERR_SEMANTIC`	Correct surface form, wrong meaning — paraphrase that inverts, weakens, or subtly misrepresents the intended claim.

Each class requires a distinct detection strategy. **No single verifier can detect all classes.** This is the foundational motivation for treating verification as a first-class architectural concern rather than a single post-processing step, and it is why a verification boundary (§3.1) is declared per class rather than as a single bit.

> **Scope note — adversarial contexts:** The classes above cover *intrinsic* model failures — ways the model's output deviates from Σ due to the model's own limitations. In adversarial contexts (especially agentic deployments with tool access), prompt injection produces ε = 1 outputs by exploiting the model's instruction-following behavior via untrusted content in its context. This is an *extrinsic* threat: the model may be operating correctly given its inputs while the system architecture that permitted untrusted content to reach the model as instructions is the locus of failure. Prompt injection is therefore out of scope for this intrinsic error taxonomy but requires separate threat-model analysis in any agentic deployment. Detection requires checking whether model behavior was influenced by untrusted context — a structurally different operation from verifying any class listed above.
>
> **Scope note — policy and compliance violations:** Enterprise deployments commonly encounter outputs that are factually correct, well-formed, and faithful to Σ, yet violate business rules, safety guardrails, or regulatory constraints (e.g., generating legally privileged content, recommending prohibited actions, or producing outputs in violation of data-handling policies). A hypothetical `ERR_POLICY` or `ERR_COMPLIANCE` class would capture this — but it is *extrinsic* to the model's semantic correctness: ε(y,x,z) = 0 (the output is acceptable in context) while a separate policy predicate π(y) = 1 (the output violates a business rule). This taxonomy intentionally restricts itself to *intrinsic* model errors (ε = 1). Policy enforcement is a distinct architectural layer that should be designed and tested independently of the verification boundary defined here.
>
> **Scope note — multimodal and agentic error classes:** The formalization assumes $\mathcal{X}$ is a space of text prompts and $\mathcal{Y}$ is a space of text output sequences. As LLMs increasingly process images, audio, and structured tool calls, the error taxonomy may require extension — notably an `ERR_TOOL_USE` class for agentic deployments where the model selects the wrong tool, fabricates tool output, or misinterprets tool results. The existing prompt-injection scope note (above) addresses the *extrinsic* agentic threat; `ERR_TOOL_USE` would address *intrinsic* tool-selection errors. Additionally, the verification cost model is unaddressed: for some error classes (e.g., `ERR_HALLUCINATION` requiring source retrieval), verification cost may exceed generation cost. These are acknowledged as out of scope for this version but are expected to be relevant for future extensions.

---

6. Argument Sketch

The following arguments support the existential form (§3.3) and motivate the operative form (§3.2). They are informal arguments, not formal proofs. Each is labeled accordingly.

6.1 Probabilistic generation is not deterministic truth

[Informal argument — empirical grounding in §4.]

All $M \in \mathcal{M}$ generate tokens by sampling from a learned conditional distribution:

$$P_\theta(y \mid x) = \prod_{t=1}^{|y|} P_\theta(y_t \mid y_{<t}, x)$$

No learned distribution over a discrete vocabulary exactly matches any ground-truth semantic specification Σ on all inputs. Were it to do so, the model would constitute a solution to arbitrary natural language understanding — a problem with no known finite-parameter solution for the full space $\mathcal{X}$.

This establishes: $\exists\, x$ such that $P_\theta(\varepsilon = 1) > 0$.

6.2 Finite parameters cannot encode unbounded world knowledge

[Informal argument. The cardinality claim below uses pigeonhole reasoning; a rigorous formalization would require defining a mapping from parameter configurations to representable propositions and bounding its image, which is not done here. Xu, Jain & Kankanhalli (2024), cited in §4.1, provide the rigorous version via computability-theoretic diagonalization.]

The parameter count $|\theta|$ is finite. The set of true factual propositions over which Σ is defined is effectively unbounded (and dynamic — facts change). The number of distinct representable belief states in θ is bounded by the model's capacity. Given that the set of true propositions is unbounded, there exist true propositions that are either unrepresented or misrepresented in θ. On inputs that invoke those propositions, $P(\varepsilon = 1) > 0$ by construction.

6.3 Evaluation is not generation

[Informal argument — empirical grounding in Kadavath et al. (2022), cited in §4.]

Even if M has encoded a correct belief about $\Sigma(x)$, the sampling process that produces y may not faithfully surface that belief. This is precisely the calibration failure class (`ERR_CALIBRATION`) and is empirically documented in the LLM literature (Kadavath et al., 2022, cited in §4).

---

7. The Capability-Detection Asymmetry

[Labeled hypothesis — not derived formally. This section states the claim, gives the operational definitions that make it measurable, and specifies the experiment that would test it. Until that experiment is run, the asymmetry is a hypothesis with a concrete supporting observation, not a result.]

7.1 Statement

As model capability $C(M)$ increases, two things happen simultaneously and in opposite directions:

1. **Low-stakes error classes become easier to detect.** `ERR_OMISSION`, `ERR_SCHEMA`, `ERR_TRUNCATION`, and `ERR_INSTRUCTION` failures tend to be structurally obvious. More capable models make these errors less frequently, and the errors they do make remain detectable by simple structural checks (missing fields, broken schemas, truncated output, violated constraints).
2. **High-stakes error classes become harder to detect.** `ERR_HALLUCINATION`, `ERR_SYCOPHANCY`, `ERR_SEMANTIC`, `ERR_CALIBRATION`, and `ERR_REASONING` failures become *more plausible* as model capability grows. A less capable model hallucinating a citation produces a nonsensical author name, a fake journal, and a malformed DOI — trivially detectable. A more capable model hallucinating a citation produces a real author name, a real journal, a plausible DOI, and an abstract that reads like the paper it should be. Detection now requires independently retrieving and reading the source — a fundamentally higher-cost operation. The same dynamic applies to calibration (confident errors become indistinguishable from confident correctness) and reasoning (multi-step inferences become harder to audit as the steps become individually more plausible).

Let $D_c(M)$ denote the detection difficulty of error class c for model M, and $C(M)$ denote model capability. For structural classes ($c \in$ {`ERR_OMISSION`, `ERR_SCHEMA`, `ERR_TRUNCATION`, `ERR_INSTRUCTION`}), D_c is approximately constant or decreasing in $C(M)$. For semantic/epistemic classes ($c \in$ {`ERR_HALLUCINATION`, `ERR_SEMANTIC`, `ERR_SYCOPHANCY`, `ERR_CALIBRATION`, `ERR_REASONING`}), D_c is increasing in $C(M)$.

Hypothesis (Capability-Detection Asymmetry):

$$\frac{\partial D_c}{\partial C} \leq 0 \quad \text{for } c \in \{\texttt{ERR\_OMISSION}, \texttt{ERR\_SCHEMA}, \texttt{ERR\_TRUNCATION}, \texttt{ERR\_INSTRUCTION}\}$$

$$\frac{\partial D_c}{\partial C} > 0 \quad \text{for } c \in \{\texttt{ERR\_HALLUCINATION}, \texttt{ERR\_SEMANTIC}, \texttt{ERR\_SYCOPHANCY}, \texttt{ERR\_CALIBRATION}, \texttt{ERR\_REASONING}\}$$

The supporting observation is qualitative but concrete: a less capable model producing a nonsensical citation (detectable by inspection) vs. a more capable model producing a plausible-but-fabricated citation with real metadata (detectable only by source retrieval). The derivative is positive by construction in that progression — the rigorous test requires the operationalization below.

7.2 Operational Definitions

Both functions are defined relative to fixed, versioned instruments, so that a change in the measurement cannot be mistaken for a change in the model.

- **Capability.** $C(M) \in [0, 1]$ is the score of M on a fixed, versioned benchmark suite $\mathcal{S}$ whose tasks lie in $\mathrm{dom}(\Sigma)$ (for example, a pinned subset of SWE-bench Verified or a pinned IFEval snapshot). The suite, its version, and the harness configuration used to run it (§4.2) are part of the definition; a score without them is not a value of C.
- **Detectability.** For a class c, fix a reference verifier V_c^* and hold it constant across every model compared. Then $D_c(M) := 1 - R_c(V_c^*, M, \mathcal{D})$, the miss rate of the fixed verifier on class-c errors produced by M (recall R_c as defined in §3.1). A class becomes *harder to detect* exactly when a verifier that used to catch its errors catches fewer of them.
- **Prevalence.** $\pi_c(M)$ is the rate at which class-c errors occur in M's output on $\mathcal{D}$ — how often the error is made, as distinct from how often it is caught.
- **Severity.** sev_c is the consequence weight of an escaped class-c error for the declared consumer.
- **Escaped risk.** $\rho_c(M) := \pi_c(M)\,(1 - R_c(V_c^*, M, \mathcal{D}))\,\mathrm{sev}_c$ — the weighted rate at which class-c errors reach a consumer. This, not detectability alone, is the quantity an engineering decision turns on.

The partial derivatives in §7.1 are then finite differences across a sequence of models ordered by C; the hypothesis predicts the sign of those differences per class. The hypothesis is about D_c only. It says nothing on its own about ρ_c, because a more capable configuration may make fewer class-c errors even as a fixed verifier catches a smaller share of them.

7.3 Experiment Protocol

1. **Fix the instruments.** Choose a realistic $\mathcal{D}$ (per the §3.2 working definition), a suite $\mathcal{S}$, and one reference verifier V_c^* per class. Freeze their versions before any model is evaluated.
2. **Fix the oracle.** Ground-truth labels $\varepsilon_c(y, x)$ must come from an oracle independent of both M and V_c^* — human adjudication or retrieval against an external source — otherwise recall is measured against the verifier itself.
3. **Size the sample per class.** As C rises, class-c errors become rarer, so a fixed number of prompts yields fewer positives for the recall estimate. Collect at least

$n_{\min}$ labeled class-c errors per model, with $n_{\min}$ chosen so that the confidence interval on R_c is narrower than the effect size the test is meant to detect. A recall estimated from a handful of positives is noise.

4. **Estimate prevalence per class per model.** Measure $\pi_c(M)$ on the same labeled sample, so that escaped risk can be computed rather than inferred.
5. **Estimate and test.** For each model, compute $C(M)$, $D_c(M)$ and $\rho_c(M)$ for every class. The detectability sign test is a rank correlation of D_c against C per class, with sign predicted by §7.1. **The reported quantity is ρ_c**; the sign test establishes the hypothesis, and ρ_c is what a boundary re-evaluation (§9.6) acts on.

Falsification: a semantic class for which D_c decreases as C increases under a fixed verifier — that is, a more capable model whose hallucinations, sycophancy, semantic drift, calibration errors, or reasoning failures are *easier* for the same verifier to catch — refutes the corresponding inequality. A structural class for which D_c increases refutes the other.

7.4 Engineering Consequence

If the asymmetry holds, then **a verification system calibrated on outputs from a less capable model is not conservative when applied to a more capable model — it is blind in precisely the highest-risk dimensions.** The errors that slipped through the old verifier were low-stakes. The errors that slip through the same verifier on a new model are the ones that cause real damage.

This is the basis of Corollary 5 (§9.6): treating the boundary as a stable component across a solver-configuration change is an architectural regression, and a boundary **re-evaluation** is the precondition. A verifier upgrade follows only where the re-evaluation shows escaped risk ρ_c rising — a more capable configuration can lower ρ_c while D_c worsens, if prevalence π_c fell faster.

---

8. Limitations

The following are honest limitations of the current formalization. They scope the specification's precision without invalidating it. Readers should have these in view before applying the corollaries (§9) or the practitioner artifacts (§10).

8.1 The error predicate ε is not computable in general

For `ERR_HALLUCINATION`, determining whether a claim is false requires access to ground truth, which is not always available. The specification asserts the *existence* of errors, not a general algorithm for finding them.

8.2 $\delta_{M,\mathcal{D}}$ is configuration-, task- and distribution-dependent

The operative form (§3.2) asserts a measured bound for a given configuration and distribution; it provides no universal constant. Calibrating $\delta_{M,\mathcal{D}}$ for a specific deployment requires: (a) a concrete characterization of the task distribution (see the working definition of "realistic distribution" in §3.2), and (b) empirical measurement on that distribution for the target solver configuration. Neither is provided by this document.

8.3 The pipeline hazard condition is unestimated

§3.4 makes no independence assumption: it decomposes pipeline risk by the chain rule and states the bound under an explicit condition — that every conditional error hazard $P(E_i \mid E_1^c, \ldots, E_{i-1}^c)$ stays at or above δ. That condition is what a deployment must estimate, and this specification does not estimate it for any pipeline.

Shared context moves the hazard in both directions. An upstream error that corrupts downstream context raises it; a downstream verifier, a retry, or a self-correcting step lowers it, and a sufficiently fast decay leaves total pipeline error bounded below 1. Where the condition is not established, §3.4's limit does not apply and residual risk must be measured directly rather than inferred from pipeline length.

8.4 Verification coverage vs. verification depth is unresolved

The contract checklist (§10.1) identifies *which error classes* a boundary covers, but does not specify the detection power within each class. A schema verifier that checks only top-level key presence has lower recall on `ERR_SCHEMA` than one that performs full recursive type validation. The checklist establishes scope; depth is a separate engineering question, and the recall R_c defined in §3.1 is the quantity that would resolve it.

8.5 The Boolean error predicate is a deliberate simplification

The predicate $\varepsilon(y, x, z) \in \{0, 1\}$ treats all errors as equivalent regardless of severity. Production verifiers almost always implement graded evaluation — a hallucinated statistic that is slightly off is a different risk level from a hallucinated statistic that inverts a safety-critical conclusion.

The binary choice is defensible for the specification's primary purpose: establishing that verification boundaries are mandatory. The binary predicate is *conservative* — it counts any unacceptability under A, however minor, as $\varepsilon = 1$. For the architectural conclusion (silent acceptance is a design defect), this is sufficient. For quantifying $\delta_{M,\mathcal{D}}$ or designing severity-weighted verification systems, a graded predicate $\varepsilon(y, x, z) \in [0, 1]$ is the appropriate extension. The operative form (§3.2) should then be read as a lower bound on expected weighted error, with weighting scheme left to the

deployment context. This extension is not formalized here and is a known gap in the current specification.

8.6 Computability-theoretic impossibility vs. practical non-negligibility

The theoretical foundations cited in §4.1 establish that the *set* of inputs on which any LLM must err is infinite. Suzuki et al. (2025) prove the complementary result: the *probability mass* on those inputs, under a given distribution, can be made arbitrarily small with sufficient training data quality and quantity. These results are not contradictory — they operate at different levels of analysis.

The operative form (§3.2) is an *empirical* claim: δ is non-negligible for all extant models on all realistic distributions. It does not rest on the computability-theoretic impossibility. If future training regimes and data quality were to drive δ below any measurement threshold for a specific restricted domain, the operative form would not apply to that domain under that model — but the existential form (§3.3) would still hold, and the architectural prescription would remain justified as a conservative design principle (it is cheaper to verify than to prove negligibility).

The practical relevance of Suzuki et al.'s result depends on the closed-world assumption: the training distribution must adequately represent the deployment distribution. Under the open-world assumption — where deployment inputs are not bounded by training coverage — the probability mass on failure inputs cannot be driven arbitrarily low. Real deployments typically operate under mixed conditions, making the distinction between closed-world and open-world a deployment-specific engineering judgment rather than a settled theoretical question.

8.7 The asymmetry is untested and the harness evidence is narrow

The Capability-Detection Asymmetry (§7) has not been measured under the protocol of §7.3; its support is one qualitative progression. The harness-level evidence in §4.2 is three studies: Lewis (2026) measures one context policy on coding tasks under one context-window budget, and the two self-improvement studies measure agents that edit their own harness, which is a narrower setting than the general deployment the Verification Boundary Principle addresses. Corollary 6 (§9.7) generalizes from those settings by argument, not by measurement. A reader who needs the corollary to hold in a specific system should treat it as a design principle to be verified there, not as an established result.

The asymmetry, if confirmed, does not by itself imply rising escaped risk. It is a claim about D_c under a fixed verifier; ρ_c also depends on prevalence π_c, which a more capable configuration may reduce faster than detectability degrades. Establishing the hypothesis would not establish that any deployed system became more dangerous.

---

9. The Verification Boundary Principle

9.1 Statement

Every system that consumes LLM output MUST declare a verification boundary $B = (S, \{V_c\}_{c \in S})$ (§3.1) that the output crosses before it reaches any consumer. The declaration MUST state:

1. **Scope** — which error classes $S \subseteq \mathcal{C}$ are checked, and, for every class in $\mathcal{C} \setminus S$, that it is an accepted risk (with a mitigation note or a downstream boundary that covers it). $S = \emptyset$ is silent acceptance and no mitigation note excuses it (§10.1).
2. **Mechanism** — what each verifier V_c is, so that its recall can in principle be measured (§8.4).
3. **Calibration** — the `SOLVER_CONFIGURATION_ID` the boundary was calibrated against, because the boundary's adequacy is a function of M (§7).
4. **Location** — where the verifier runs, and that no producer (the model, or any agent it controls) can write to it (§9.7).
5. **Specificity** — the false-positive rate of each verifier alongside its recall. Recall alone is inadequate: a verifier that rejects every output has recall 1 and is useless.
6. **Failure behaviour** — what happens when a verifier rejects, per class: retry, abstain, escalate, or fall back.
7. **Oracle** — the evidence source each verifier checks against.
8. **Severity and residual risk** — sev_c per class, and the resulting severity-weighted residual risk $\sum_c P(\varepsilon_c)\,(1 - R_c)\,\text{sev}_c$.
9. **Tolerated failure rate** — τ for the declared consumer (§3.2).
10. **Ownership** — the boundary owner and the calibration date.

A compliant boundary demonstrates, for the declared consumer and consequence, that the residual risk of the output crossing it is acceptable. LLM output remains untrusted until that demonstration exists.

The declaration is an artifact (§10), reviewable without running the system. A system without such a declaration exhibits silent acceptance and contains an architectural defect, whatever its observed output quality.

The following corollaries are direct implications of the invariant (§3), the taxonomy (§5), and the asymmetry (§7). They are not recommendations — they are logical consequences, with the epistemic status of the claims they rest on.

9.2 Corollary 1 — Appearance of correctness is not correctness

A system that validates LLM output by inspection on a finite test set has not demonstrated error-freedom. It has demonstrated error-absence on the tested inputs. The unverified long tail of $\mathcal{X}$ retains non-zero error probability.

Practical implication: Manual review during development does not substitute for runtime verification in production.

9.3 Corollary 2 — Trust accumulation is prohibited

Observing correct outputs on inputs $x_1, \ldots, x_k$ provides no guarantee about $P(\varepsilon = 1)$ on a new input $x_{k+1} \notin \{x_1, \ldots, x_k\}$. The operative form's bound holds over the distribution, not over history. No finite sequence of correct outputs establishes zero residual risk or justifies relaxing a boundary outside a predefined statistical recalibration procedure. Observed successes may update an *estimate* of $\delta_{M,\mathcal{D}}$; they do not remove the boundary.

Practical implication: A boundary may be relaxed only through a recalibration procedure defined in advance — a stated estimator, sample size and acceptance threshold — never by accumulated confidence from a run of correct outputs.

9.4 Corollary 3 — Verification scope must match error taxonomy

A verifier that checks only JSON schema conformance (`ERR_SCHEMA`) does not cover hallucination (`ERR_HALLUCINATION`) or sycophantic drift (`ERR_SYCOPHANCY`). Partial verification is better than none, but must be scoped honestly — a system claiming "verified output" must declare *which error classes* its boundary covers.

Practical implication: Verification claims must be scoped and documented, not asserted globally. This is requirement 1 of §9.1.

9.5 Corollary 4 — Silent acceptance is an architectural defect

Any production system that passes LLM output directly to downstream consumers without a declared verification boundary has an architectural omission. This defect exists regardless of observed output quality and regardless of model capability. It is structural, not runtime.

Practical implication: Code review should treat silent acceptance as a blocking defect, equivalent to missing authentication on a sensitive endpoint. The check is mechanical — a call site whose output flows to a consumer with no boundary declaration in scope — and can be automated (§10.5).

9.6 Corollary 5 — A boundary re-evaluation is a precondition for a solver-configuration change

If the Capability-Detection Asymmetry (§7) holds, a boundary calibrated against configuration M_1 has lower recall on the semantic classes when applied to a more capable M_2. Recall is not the reported quantity, however: what matters is escaped risk $\rho_c(M) = \pi_c(M)\,(1 - R_c)\,\mathrm{sev}_c$ (§7.2). A new configuration may *lower* ρ_c even while R_c falls, because prevalence π_c fell faster. The consequence is therefore conditional, and an unconditional "upgrade the verifier" rule would be wrong.

Practical implication: A boundary re-evaluation is a precondition for any solver-configuration change. A verifier upgrade is required whenever that re-evaluation

shows ρ_c rising for any class. Treating the boundary as a stable component across a configuration change is an architectural regression. This is why the contract block (§10.1) pins `SOLVER_CONFIGURATION_ID`.

9.7 Corollary 6 — The acceptance authority must remain outside the producer's control domain

A verifier V_c that the model, or any agent the model controls, can modify is subject to two failures at once. First, it inherits the model's error distribution: an edit to V_c is itself LLM output, and by §3.2 it is wrong with non-negligible probability. Second, it is under optimization pressure: when acceptance is decided by a score the system can influence, an edit that weakens the check is rewarded exactly as an edit that improves the work. Wang et al. (2026) observe such edits in real self-improving runs and find that they persist in the lineage of the best-performing agent; Guo et al. (2026) show self-authored verification scores staying near perfect while sealed deployment performance degrades, and find that at least one acceptance signal outside the agent's control is required to close the gap (§4.2).

Practical implication: The boundary B contains the verifiers, so the requirement is not that they sit outside B — it is that they sit outside the *producer's control domain*. No producer (the model, or any agent it controls) may write to a verifier or to the record of its verdicts. In practice: a separate process or privilege domain, versioned and reviewed like production code, with verdicts appended where a party outside the runtime can check them. This is requirement 4 of §9.1, declared as `ACCEPTANCE_AUTHORITY` in §10.1. A "verification" step the agent can rewrite is not a boundary; it is silent acceptance with extra steps.

---

10. Practitioner Artifacts

Copy-paste artifacts for declaring a verification boundary at the function level (§10.1–10.3), at the project level (§10.4), and in continuous integration (§10.5). All artifacts in this section are released under CC0 1.0 so that they can be pasted into any codebase without attribution.

10.1 Full Contract Block

For any function that invokes an LLM:

```
/**
 * (!) VERIFICATION BOUNDARY — Silent Acceptance specification
 *
 * SILENT_ACCEPTANCE_VERSION: 2.1.0
 *   ^ REQUIRED. Spec version this boundary was declared against
 *   (concept DOI 10.5281/zenodo.19401266). A spec update without boundary
```

```
*   review may introduce new error classes or change requirements.
*
* SOLVER_CONFIGURATION_ID: <model + harness + context policy + tools + prompts>
*   ^ REQUIRED. §7 (Capability-Detection Asymmetry) hypothesizes that
*   verification requirements shift when capability changes, and §4.2 shows the
*   harness moves measured capability on fixed weights. This boundary is valid
*   only for the pinned configuration; a configuration change without boundary
*   re-evaluation is an architectural regression (§9.6).
*
* VERIFIER_LOCATION: <module or service that runs the verifiers below>
*   ^ REQUIRED. Must not be writable by the model or by any agent the model
*   controls (§9.7). If the verifier lives in the same process as an agent
*   that can edit code, say so here and name the control that prevents edits.
*
* ACCEPTANCE_AUTHORITY: <where verdicts are recorded, outside the producer's
*   control domain>
*   ^ REQUIRED (§9.7). An append-only record a party outside the runtime can
*   read. If the producer can rewrite the verdict log, there is no boundary.
*
* TOLERATED_FAILURE_RATE: <τ for the declared consumer>
* OWNER: <team or person accountable for this boundary>
* CALIBRATED_ON: <YYYY-MM-DD>
*   ^ REQUIRED (§9.1 items 9-10).
*
* INVARIANT (operative form): for solver configuration M and realistic
* distribution D, the measured error rate exceeds what the consumer tolerates:
*
*   E[ε(y, x, z)] ≥ δ_{M,D} > τ    (δ_{M,D} is measured, not assumed)
*
* Existential form (formally establishable): ∃ (x,z) such that P(ε(y,x,z)=1) > 0
*
* For a pipeline of length n, by the chain rule and with no independence
* assumption:
*
*   P(pipeline error-free) = ∏ P(Eᵢᶜ | E₁ᶜ … Eᵢ₋₁ᶜ)
*   If every conditional hazard ≥ δ, then P(at least one error) → 1 as n → ∞.
*   The hazard condition is what a deployment must estimate — see §3.4, §8.3.
*
* CONSEQUENCE: Any caller of this function that does not explicitly
* validate the output is introducing an architectural omission —
* not a downstream bug.
*
* PER-CLASS BOUNDARY TABLE - every class carries an explicit status.
* status is COVERED or ACCEPTED_RISK: <reason>.
*
*  class             | verifier | oracle | rec/spec | on reject | status
*  ERR_HALLUCINATION |          |        |          |           |
*  ERR_OMISSION      |          |        |          |           |
```

```
 *  ERR_SCHEMA        |          |        |          |           |
 *  ERR_TRUNCATION    |          |        |          |           |
 *  ERR_SYCOPHANCY    |          |        |          |           |
 *  ERR_INSTRUCTION   |          |        |          |           |
 *  ERR_CALIBRATION   |          |        |          |           |
 *  ERR_SEMANTIC      |          |        |          |           |
 *  ERR_REASONING     |          |        |          |           |
 *
 * Specificity is required alongside recall: a verifier that rejects
 * everything has recall 1 (§9.1 item 5). "on reject" is retry | abstain |
 * escalate | fallback (item 6).
 *
 * If no row is COVERED then S = ∅. That is silent acceptance, and no
 * mitigation note excuses it — the linter reports it as an error (§10.5).
 */
```

10.2 Short-Form (headers, PR descriptions, commit messages)

```
VERIFICATION BOUNDARY REQUIRED (Silent Acceptance v2.1.0):
Measured LLM error rates exceed tolerated failure rates across realistic
deployments. Pin the SOLVER_CONFIGURATION_ID; a boundary covering no class
(S = ∅) is silent acceptance, whatever mitigation is noted.
Passing LLM output onward with no declared verification boundary is a
design defect, not a runtime bug. All LLM output must be treated as
untrusted and validated explicitly, per error class.
```

10.3 Inline Banner (single-line, for code comments)

```
// (!) VERIFICATION BOUNDARY: LLM output is untrusted by default.
// Verify before use; S = ∅ is silent acceptance. (Silent Acceptance v2.1.0)
```

10.4 Agent Instruction File Block (AGENTS.md, CLAUDE.md, and equivalents)

Paste verbatim into the repository's agent instruction file. `AGENTS.md` is the cross-vendor convention read by most coding agents; vendor-specific files such as `CLAUDE.md`, `GEMINI.md`, or `.cursor/rules` may include it by reference or carry the same block. The block is identical for every tool.

```
## LLM Output Verification — Architectural Requirement (Silent Acceptance)

**SILENT_ACCEPTANCE_VERSION:** 2.1.0
**Specification:** https://doi.org/10.5281/zenodo.19401266

**Across any realistic deployment, LLMs reliably produce errors —
at a rate that is non-zero and non-negligible.**
For any model M and realistic task distribution D:
```

```
    E[ε(M(x), x)] ≥ δ > 0   (δ non-negligible, empirically measurable)

No individual call is guaranteed to fail; the distribution over calls
guarantees failure at scale. Omissions, hallucinations, partial completions,
schema violations, and silent failures are not edge cases — they are
statistical properties of the model class. In pipelines of length n
(assuming approximately independent calls — see full specification for
correlation analysis), unverified error probability approaches 1
monotonically.

Failing to verify LLM output is not a bug in the generated artifact.
It is an **architectural omission** in the system that consumed it.

Every pipeline, agent, or workflow that accepts LLM output MUST treat
that output as **untrusted, incomplete, and unverified by default**, and
MUST declare a verification boundary: which error classes are checked,
by what, before the output reaches a consumer. Verification is not optional
post-processing — it is a first-class design concern, on par with
authentication and input validation.

Rules for agents working in this repository:

1. Every call site that invokes an LLM and passes its output onward carries a
   verification boundary declaration (the contract block in the specification,
   §10.1) or a reference to the boundary that covers it.
2. The verifier is never edited by the agent whose output it judges.
   Proposed changes to a verifier go to a human reviewer.
3. A model version change is a boundary review, not a config change.

> Silent acceptance — passing LLM output onward with no declared verification
> boundary — is a design defect, regardless of how correct the output appears.

Full specification: [SILENT_ACCEPTANCE.md](./SILENT_ACCEPTANCE.md)
```

10.5 CI Check

The companion linter `silent-acceptance-lint` mechanizes Corollary 4 (§9.5). It scans source files for LLM call sites and reports each one that has no boundary declaration in scope. The check is a *presence* check on the declaration, not a dataflow analysis: it cannot prove that a declared verifier is adequate (§8.4), only that a boundary was declared and that the declaration is not empty. That is the reviewable artifact the principle requires; adequacy is the job of the reviewer and of the recall measurement in §7.3.

What it flags:

- an LLM call site with no `SILENT_ACCEPTANCE_VERSION:` (or `@verification-boundary`) declaration within the enclosing scope window or at file level;

- a declaration whose checklist leaves every error class unchecked and gives no `MITIGATION:` note (the blocking case in §10.1);
- a declaration with no `SOLVER_CONFIGURATION_ID:` pin (`MODEL_VERSION` is accepted as a deprecated alias and reported as a warning);
- a declaration with no `ACCEPTANCE_AUTHORITY:`;
- a per-class table in which no row is `COVERED` ($S = \emptyset$).

The tool and its tests live in the `silent-acceptance-lint/` directory of the specification's repository (§11). It has no runtime dependencies and runs on Node 24 directly from a checkout:

```
node silent-acceptance-lint/src/cli.ts src/
```

GitHub Actions, checking the tool out beside the project:

```yaml
name: verification-boundary
on: [push, pull_request]
jobs:
  silent-acceptance:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/checkout@v4
        with:
          repository: pedroanisio/silent-acceptance
          path: .silent-acceptance
      - uses: actions/setup-node@v4
        with:
          node-version: 24
      - run: node .silent-acceptance/silent-acceptance-lint/src/cli.ts src/
```

A boundary declaration is anything the linter recognizes as one: the contract block of §10.1, or a comment carrying `SILENT_ACCEPTANCE_VERSION:` or `@verification-boundary`. A call site may be excused with `silent-acceptance-ignore: <reason>`; an excuse without a reason is itself a finding.

---

11. Provenance, Acknowledgments, and Version History

This section is the document's own verification boundary: it states what was checked, by what, and what was not, so that a reader can decide which claims to rely on without taking any of them on trust.

11.1 What was verified, and how

- **References.** Every entry in §4, §4.1, and §4.2 carries a DOI or an arXiv identifier. The companion audit tool (`pals-check`, in the repository named in §11.5) resolves each identifier over the network and records the outcome per reference — the

fetched title, the URL it resolved to, and whether the claimed title matched — in `pals_law_report.json`, which is published alongside this document under the same DOI. The Note column of each reference table states how the relevance claim was checked and when.

- **Mathematics.** Results attributed to a published theorem cite it (§4.1). Every other derivation is labeled at the point where it appears — *informal argument* in §6.1–6.3, the explicit convergence condition in §3.4 — and is a first-principles derivation, not a theorem. The companion audit records its consistency checks on the display mathematics (operative-form structure, existential weakening, pipeline algebra and convergence condition, autoregressive factorization, asymmetry signs and error-class coverage, cross-reference integrity) in the same report.
- **Hypotheses.** The Capability-Detection Asymmetry (§7) is labeled a hypothesis and has not been tested under its own protocol (§7.3, §8.7). Corollary 6 (§9.7) generalizes from three studies by argument, not by measurement (§8.7).
- **Not verified.** The working definition of "realistic distribution" (§3.2) is a characterization, not a formal definition, and δ has not been measured for any specific deployment (§8.2). Coverage of the nine error classes by dedicated empirical references is five of nine (§4, coverage note).

11.2 Generation disclosure

This document was drafted by the author with AI assistance. Versions 1.x were drafted with Claude Sonnet 4.6 via claude.ai and typeset with Claude Opus 4.6 (April 2026). Version 2.0.0 was drafted, restructured, and tooled with Claude Fable 5.1 via Claude Code on 2026-09-03. The author is responsible for every claim.

11.3 On the name

Versions 1.x carried the author's name. Eponymous engineering laws are conferred by others — Hyrum's Law was named by Titus Winters, not by Hyrum Wright (https://www.hyrumslaw.com/) — and a self-conferred eponym asks the reader to grant status before the idea has earned it. The v2 title names the defect the specification identifies; the author's name belongs in the citation, where it now is.

11.4 Review notes

The September 2026 review (v2.1.0). A structured review raised seven blocking issues. Six were adopted: the semantic specification Σ replaced by an acceptability predicate $A(y, x, z)$ carrying an evaluation context (A1); δ indexed to configuration and distribution and given an operational threshold τ (A2); the independence-based product formula replaced by a conditional-hazard decomposition with its condition stated (A3); the theoretical citations narrowed to existential risk (B1); Corollary 2 reworded away from an inductive-impossibility claim (C3); and Corollary 6 restated as a control-domain requirement (C5). One was adopted in part: the Capability-Detection

Asymmetry keeps its detectability claim, the “migration” language is removed, and escaped risk ρ_c is introduced so that the engineering consequence is conditional on prevalence rather than on detectability alone (C4).

One recommendation was **declined**. The review proposed repositioning the invariant as a per-deployment risk argument rather than a premise. It is retained as a premise because that is what makes silent acceptance a *default* defect: if the invariant must be argued deployment by deployment, the burden falls on whoever asks for a boundary rather than on whoever omits one, which inverts the specification’s purpose. The assurance-case formulation was adopted instead as the statement of what a compliant boundary demonstrates (§9.1), which answers the review’s substance without moving the burden.

The review’s provenance is stated honestly: it was AI-assisted, produced by the same class of process that drafted the document, and its recommendations were evaluated rather than adopted wholesale.

The rename, the promotion of §7, the harness evidence in §4.2, the tool-agnostic artifacts, and the placement of this section respond to an earlier review produced in September 2026 by the same AI-assisted process that drafted the document (§11.2), not by an independent reviewer. Its recommendations were evaluated rather than adopted: the asymmetry is placed after the taxonomy it depends on rather than before it, as the review proposed, and the universal-form note in §3.3 was compressed rather than removed because it answers the objection that the invariant is trivially true. Peer review by people who did not take part in drafting has not yet happened.

11.5 Publication, license, and citation

All versions share the Zenodo concept DOI 10.5281/zenodo.19401266, which always resolves to the latest version. Version 1.5.4 was uploaded three times on 2026-04-03; the last of them, Zenodo version 3, is record 10.5281/zenodo.19401530 and carries the PDF and the audit artifacts (the earlier uploads are records 19401267 and 19401346). The source, the spec audit tool `pals-check`, and the code-side linter `silent-acceptance-lint` are maintained at https://github.com/pedroanisio/silent-acceptance.

The specification text is released under CC BY 4.0. The practitioner artifacts in §10 are released under CC0 1.0.

```
@misc{silent_acceptance_2026,
  author       = {de Luna e Silva, Pedro Anisio},
  title        = {Silent Acceptance: {LLM} Output Error as an Architectural
  Invariant},
  year         = {2026},
  version      = {2.0.0},
  doi          = {10.5281/zenodo.19401266},
  note         = {Formerly published as PALS's Law (v1.x). Version 2.0.0, September
  2026}
}
```

11.6 Version history

- **v2.1.0** — Replaced the semantic specification Σ with an acceptability predicate $A(y, x, z)$ over an evaluation context; indexed the operative bound as $\delta_{M,\mathcal{D}}$ against a declared tolerance τ; replaced the independence product with a conditional-hazard decomposition and stated its condition; narrowed the theoretical citations to existential risk and added Suzuki et al. to §4.1; corrected the Kadavath and Berglund evidence rows; defined solver configurations and replaced `MODEL_VERSION` with `SOLVER_CONFIGURATION_ID`; extended §9.1 to ten declaration fields including specificity, failure behaviour, oracle, severity-weighted residual risk, τ and ownership; added the assurance-case statement; reworded Corollary 2; restricted §7 to detectability and introduced prevalence π_c and escaped risk ρ_c; restated Corollary 5 as boundary re-evaluation and Corollary 6 as a control-domain requirement; replaced the contract checkbox list with a per-class table in which $S = \emptyset$ is blocking with no mitigation escape; bumped silent-acceptance-lint accordingly.
- v1.1 — Demoted unestablished strong form; elevated operative form as primary claim.
- v1.2 — Added working definition of "realistic distribution"; added independence caveat; added `MODEL_VERSION` to contract block.
- v1.3 — Restructured section order to match logical dependency graph; merged practitioner artifacts; corrected stale cross-references.
- v1.4 — Added `ERR_REASONING` to taxonomy; added prompt-injection scope note; added Sharma et al. (2023); disclosed Boolean predicate simplification; formalized the capability-detection asymmetry as a labeled hypothesis.
- v1.5 — Added positioning paragraph and theoretical foundations (Kalai & Vempala, Xu et al., Karpowicz, Suzuki et al.); completed 9-class taxonomy coverage; added computability-theoretic vs. practical non-negligibility discussion; added Berglund et al. and Zhou et al. (IFEval) as empirical anchors; tightened dom(Σ) restriction and Corollary 2; added scope notes for `ERR_POLICY`, `ERR_TOOL_USE`, and verification cost model.
- v2.0.0 — Renamed from *PALS's Law* to *Silent Acceptance*; named the prescription the Verification Boundary Principle (§9) and gave the verification boundary a definition (§3.1); promoted the Capability-Detection Asymmetry from a corollary to its own section with operational definitions and an experiment protocol (§7); added harness-level evidence (§4.2) and Corollary 6, the verifier must sit outside the boundary it verifies (§9.7); compressed the universal-form note (§3.3) to its two load-bearing sentences; made the practitioner artifacts tool-agnostic (§10.4 agent instruction file block, §10.5 CI check) and renamed the contract field to `SILENT_ACCEPTANCE_VERSION`; replaced the front-of-document disclaimer with the verification statement in §11.1 and moved the generation disclosure and version history here; added §8.7 on the limits of the new evidence.

End of document — Silent Acceptance v2.0.0